The Journal of Open Source Software

CEEDesigns.jl: Cost-Efficient Experimental Design in Julia

Jan Bíma [1], **Tianchi Chen** [2], **Kamala Dadashova** [2¶], **Gwenyth Jones**[2], **Otto Ritter**[1], **and Sean L. Wu** [2]

1 DSSI Decision Science, MSD Czech Republic **2** DSSI Decision Science, Merck & Co., Inc., Rahway, NJ, USA ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Richard Liu

Reviewers:

- @himaghna
- @apraga

Submitted: 07 July 2026
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

`CEEDesigns.jl` is a Julia package for cost-efficient design of experiments. A design consists of one or more experiments; each experiment is an action to acquire additional empirical evidence which incurs a monetary cost and time to execute. Given a set of experiments and historical results, `CEEDesigns.jl` solves for the Pareto-efficient front of designs that balance the value of the evidence they yield against the execution cost they incur.

To find efficient designs for a population of experimental entities, `CEEDesigns.jl` implements two design regimes. The *static* regime fixes one design for an entire population. It estimates the information value of each possible design from the cross-validated performance of a predictive model trained with `MLJ.jl` (Blaom et al., 2020), then schedules the chosen subset to minimize execution cost. The *dynamic* regime builds a design one experiment at a time for a single entity. Selection of the next experiment is conceived of as a Markov decision process (Egorov et al., 2017). Each iteration simulates an experimental run, updates beliefs about unobserved outcomes, and accumulates costs. The design is solved online with Monte Carlo tree search (Browne et al., 2012; Sunberg & others, 2024).

Both regimes learn what they need directly from historical data: a predictive model in the static case, a similarity-weighted resampling of past data in the dynamic one. Neither requires a mechanistic simulator. `CEEDesigns.jl` was developed for assay triage optimization in drug discovery (Chen et al., 2026), but applies in any domain where experiments incur real costs, historical analogs are available, and resources must be spent to reduce uncertainty to an acceptable level.

Statement of Need

In drug discovery, thousands of candidates advance through iterative cycles of design, synthesis, and testing to find promising pre-clinical candidates. During testing, compounds are evaluated for potency, bioavailibilty, safety, and other features on assays that span orders of magnitude in cost and predictiveness, from $100-scale *in vitro* readouts to definitive *in vivo* endpoints costing thousands of dollars and several weeks (Segall, 2012). Resource constraints reframe the operative question from *can we fully characterize this candidate?* To *what is the smallest experimental sequence that resolves the decision at proportionate cost?*

Assay triage for compounds occurs continuously: beliefs must be updated as each readout arrives, correlation among assays exploited, downstream consequences of each choice anticipated, and total spend justified against information gained. In practice, scientists rely on inflexible rules, leading to policies that tend to be risk-averse or short-horizon, leaving avoidable cost on the table.

DRAFT

`CEEDesigns.jl` recasts this triage as a decision problem and delivers it as a Julia package, aimed at computational chemists, assay scientists, and decision-science teams who must prioritize experiments under tight budgets. The same abstractions transfer to any setting where measurements are costly, historical analogs exist, and no mechanistic simulator is available.

State of the Field

The cost-aware, evidence-adaptive triage problem draws on several mature literatures. Bayesian experimental design (BED) selects experiments that maximize expected information gain (Chaloner & Verdinelli, 1995; Huan et al., 2024; Rainforth et al., 2024), but most canonical methods are *myopic* and presume an explicit likelihood or simulator; extensions to implicit likelihoods (Kleinegesse & Gutmann, 2020) and amortized policy learning (Blau et al., 2022; Foster et al., 2021; Ivanova et al., 2021) relax these assumptions only at the cost of substantial training infrastructure. Bayesian optimization (Shahriari et al., 2016), in its standard form, improves a single objective rather than balancing information against resources, and Bayesian reinforcement learning (Ghavamzadeh et al., 2015) needs either an explicit (posterior) transition model or sampled interaction trajectories (s, a, r) that pre-clinical triage rarely provides.

On the software side, the Python ecosystem offers `BoTorch` (Balandat et al., 2020) and `BoFire` (Dürholt et al., 2025) for (multi-objective) Bayesian optimization, primarily over Gaussian-process surrogates and including mixed continuous, discrete, and categorical design spaces; `Pyro` (Bingham et al., 2019) additionally ships an explicit Bayesian-optimal-design module (`pyro.contrib.oed`) for expected-information-gain estimation, and general-purpose probabilistic-programming languages such as `PyMC` (Salvatier et al., 2016) can express BED objectives, though they provide no dedicated BED tooling. These tools excel at one-shot or short-horizon optimization, but rely on myopic single-step acquisition: they do not plan multi-step sequences of discrete assays under coupled monetary and time budgets, nor build a similarity-based, simulator-free transition model from historical records.

`CEEDesigns.jl` targets that gap. It models monetary cost and execution time explicitly, plans dynamically over sequences of assays, returns a Pareto frontier rather than a single recommendation, derives its dynamic transition model nonparametrically from historical outcomes, and composes with the established Julia machine-learning and decision-making stack (Blaom et al., 2020; Egorov et al., 2017; Sunberg & others, 2024).

Software Design

`CEEDesigns.jl` is a thin composition over the Julia scientific stack (Bezanson et al., 2017) rather than a reimplementation. A top-level module supplies a shared Pareto-front primitive and plotting helpers for two submodules: `StaticDesigns` and `GenerativeDesigns` for static and dynamic designs, respectively. The two regimes are deliberately built on common parts. Each expresses the value of information as a modular uncertainty measure of a target property, experimental costs by a `(monetary, time)` pair combined under a user-set trade-off, and returns the non-dominated set in the cost–uncertainty plane. Where they conceptually differ is in how the design is *searched*.

In the **static** regime, the value of a measurement subset is the cross-validated predictive performance of an `MLJ.jl` model trained on the corresponding features, evaluated over the powerset of experiments. The chosen subset is then arranged optimally, with a Markov decision process (MDP) entering only afterwards as a heuristic search device for scheduling the already-selected experiments into efficient parallel batches. The static regime also carries an optional *filtration* term: the fraction of entities that survive a screen and proceed to follow-up experiments, which discounts the cost of all downstream work so that a screen eliminating most candidates early is costed accordingly.

DRAFT

In the **dynamic** regime, by contrast, experiment selection *itself* is the MDP, because the next-best action depends on outcomes not yet observed. The regime's distinguishing choice is a *simulator-free*, nonparametric transition model built directly from a historical `DataFrame`. Each historical row D_i receives a weight $w_i(s) = \exp(-\lambda_w\, d(s, D_i))$ from a similarity kernel applied to a distance $d(s, D_i)$ between the current evidence s and that row; transitions are then simulated by resampling rows in proportion to these weights, recomputed as evidence arrives so that belief concentrates on consistent historical analogs (Chen et al., 2026). The distance is type-aware and configurable, with a Mahalanobis option for correlated features. The MDP is a subtype of `POMDPs.MDP` solved online with Monte Carlo tree search using double progressive widening (`MCTS.jl`).

The dynamic regime exposes three planners that answer three decision framings. To drive overall uncertainty about a single target below an acceptable level at least cost, `UncertaintyReductionMDP` minimizes expected cost subject to a terminal bound on residual uncertainty. To place the target confidently inside or outside a region of interest, `ConditionalUncertaintyReductionMDP` adds a *goal-likelihood* stopping rule that halts once posterior mass in that region crosses a belief threshold, with companion routines aggregating repeated runs to quantify stability. Where there is no fixed uncertainty bar but an explicit, possibly nonlinear utility function, `EfficientValueMDP` optimizes cost of information.

A minimal static design illustrates the surface. Experiments are declared as a mapping from a name to its cost and the features it reveals; the rest composes over `MLJ.jl`, `DataFrames.jl`, `POMDPs.jl`, and `Plots.jl`:

```
using CEEDesigns, CEEDesigns.StaticDesigns, MLJ

experiments = Dict(
    "MutationPanel" => (300.0, 5.0) => ["TP53", "EGFR", "PTEN"],
    "Histology"     => (2000.0, 14.0) => ["grade_score"],
)
evals   = evaluate_experiments(experiments, model, X, y; measure = LogLoss())
designs = efficient_designs(experiments, evals; max_parallel = 2, tradeoff = (0.5, 0.5))
```

Tutorials accompany every entry point, from the introductory `SimpleStatic.jl` and `SimpleGenerative.jl` to the applied `GliomaGrading.jl` and `ConditionalUncertaintyReduction.jl`. Full API documentation is maintained in the repository.

Research Impact

`CEEDesigns.jl` originated within the Dynamics of Value Evolution (DyVE) framework for computational R&D process modeling, alongside the sibling Julia packages `ReactiveDynamics.jl` (Bíma et al., 2022) and `AlgebraicAgents.jl` (Bíma, Wu, et al., 2026) for hierarchical dynamical-system co-integration. It was presented to the scientific-computing community as a contributed talk at JuliaCon 2024 (Bíma et al., 2024), and the underlying methodology is to appear at the European Conference on Machine Learning and Principles and Practice of Knowledge Discovery in Databases (ECML PKDD) 2026 (Bíma, Chen, et al., 2026).

Two applications demonstrate the package in use. In a glioma-grading example (low-grade glioma vs. glioblastoma), zero-cost clinical features and separately costed gene-mutation assays feed a classifier; the static Pareto frontier traces classification accuracy against combined cost, with and without an optional digital-pathology endpoint, and the shift between the two upper envelopes reports the endpoint's marginal value as cost per unit of uncertainty reduction (Figure 1).

DRAFT

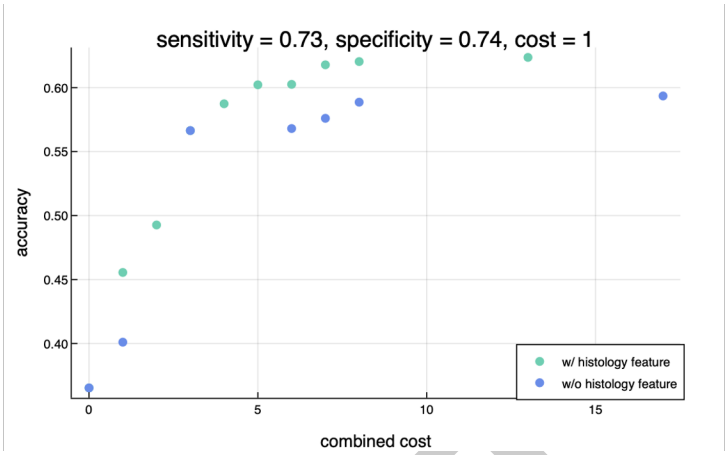


Figure 1: Glioma grading (static designs). Accuracy versus combined cost for subsets of experiments with (green) and without (blue) a simulated histology endpoint; the gap between the two upper envelopes quantifies the endpoint's marginal value.

The dynamic regime has been applied operationally at MSD to CNS brain-penetration triage, spanning several modalities—zero-cost QSAR (Quantitative Structure Activity Relationship) predictions, three *in vitro* transporter readouts (P-glycoprotein at two concentrations and BCRP; \$400 and 7 days each), and a definitive *in vivo* k_{puu} measurement (\$4,000, 21 days)—against a historical cohort of $N = 220$ fully profiled compounds. Across four representative scenarios (Figure 2), the adaptive planner matched or exceeded the rule-based baseline's decision confidence with \$400–\$800 plans, against the baseline's full \$5,200 panel: up to a 92% reduction in assay spend while preserving or improving decision confidence (Chen et al., 2026). The framework complements related cheminformatics work by some of the authors, including the `QComp` QSAR-based imputation framework (Yang et al., 2025), which can supply the zero-cost predictive features that seed a triage problem.

DRAFT

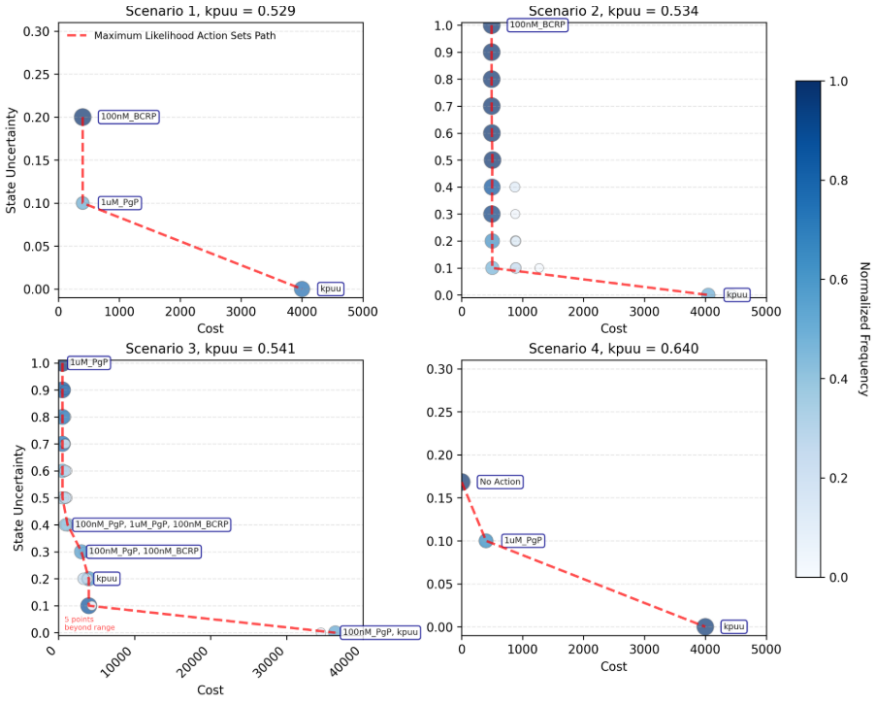


Figure 2: CNS brain-penetration triage (dynamic designs). Cost versus terminal state-uncertainty for four representative compounds; the maximum-likelihood action-set path (red) traces the planner's recommended sequence.

Availability and Documentation

`CEEDesigns.jl` is MIT-licensed and registered in Julia's General registry; it installs through Julia's built-in package manager via `Pkg.add("CEEDesigns")`. The documentation covers installation, an API reference, and the runnable tutorials cited above, which double as worked examples for both regimes. Contributions and bug reports are welcome via GitHub; see the repository's `CONTRIBUTING.md` for guidance on extending the framework with new uncertainty measures, distances, or planners.

AI Usage Disclosure

The authors used GitHub Copilot for inline code completion during package development, and Claude Opus 4 (Anthropic) for copy-editing and structural refinement of this manuscript text. No AI tool was used for the design of the algorithms, the implementation of the methods, the experimental results, or any interaction with editors or reviewers. All AI-assisted outputs were reviewed, edited, and validated by the authors, who made all core design decisions and bear full responsibility for the final work.

Acknowledgements

This research was conducted as part of the authors' employment and received no external funding. The authors are employees of subsidiaries of Merck & Co., Inc., Rahway, NJ, USA, and may hold stock and/or stock options in Merck & Co., Inc., Rahway, NJ, USA. The authors have no other competing interests to declare.

DRAFT

References

Balandat, M., Karrer, B., Jiang, D. R., Daulton, S., Letham, B., Wilson, A. G., & Bakshy, E. (2020). BoTorch: A framework for efficient Monte-Carlo Bayesian optimization. *Advances in Neural Information Processing Systems*, *33*, 21524–21538.

Bezanson, J., Edelman, A., Karpinski, S., & Shah, V. B. (2017). Julia: A fresh approach to numerical computing. *SIAM Review*, *59*(1), 65–98. https://doi.org/10.1137/141000671

Bíma, J., Chen, T., Dadashova, K., Jones, G., Ritter, O., Wu, S. L., Yang, B., & Yu, X. (2026). Cost-aware triage in drug discovery: A CEEDesigns.jl case study at MSD. In *Machine learning and knowledge discovery in databases (ECML PKDD 2026)*. Springer.

Bíma, J., Chen, T., Ritter, O., & Wu, S. L. (2024). *Cost-efficient experimental designs, aka CEEDesigns.jl*. Contributed talk, JuliaCon 2024, TU Eindhoven, Netherlands. https://pretalx.com/juliacon2024/talk/PJLF73/

Bíma, J., Wu, S. L., & Ritter, O. (2026). AlgebraicAgents.jl: Hierarchical composition of multi-formalism dynamical systems. *Journal of Open Source Software*, *11*(122), 9989. https://doi.org/10.21105/joss.09989

Bingham, E., Chen, J. P., Jankowiak, M., Obermeyer, F., Pradhan, N., Karaletsos, T., Singh, R., Szerlip, P., Horsfall, P., & Goodman, N. D. (2019). Pyro: Deep universal probabilistic programming. In *Journal of Machine Learning Research* (No. 28; Vol. 20, pp. 1–6).

Bíma, J., Ritter, O., & Wu, S. L. (2022). *ReactiveDynamics.jl: A modeling package for reaction network-type dynamical systems*. https://github.com/Merck/ReactiveDynamics.jl.

Blaom, A. D., Kiraly, F., Lienart, T., Simillides, Y., Arenas, D., & Vollmer, S. J. (2020). MLJ: A Julia package for composable machine learning. *Journal of Open Source Software*, *5*(55), 2704. https://doi.org/10.21105/joss.02704

Blau, T., Bonilla, E. V., Chades, I., & Dezfouli, A. (2022). Optimizing sequential experimental design with deep reinforcement learning. *Proceedings of the 39th International Conference on Machine Learning*, 2107–2128.

Browne, C. B., Powley, E., Whitehouse, D., Lucas, S. M., Cowling, P. I., Rohlfshagen, P., Tavener, S., Perez, D., Samothrakis, S., & Colton, S. (2012). A survey of Monte Carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in Games*, *4*(1), 1–43. https://doi.org/10.1109/TCIAIG.2012.2186810

Chaloner, K., & Verdinelli, I. (1995). Bayesian experimental design: A review. *Statistical Science*, *10*(3), 273–304. https://doi.org/10.1214/ss/1177009939

Chen, T., Bíma, J., Wu, S. L., Ritter, O., Yang, B., & Yu, X. (2026). *Case-guided sequential assay planning in drug discovery*. https://arxiv.org/abs/2601.14710

Dürholt, J. P., Asche, T. S., Kleinekorte, J., Mancino-Ball, G., Schiller, B., Sung, S., Keupp, J., Osburg, A., Boyne, T., Misener, R., Eldred, R., Kappatou, C., Lee, R. M., Linzner, D., Steuer Costa, W., Walz, D., Wulkow, N., & Shafei, B. (2025). BoFire: Bayesian optimization framework intended for real experiments. *Journal of Machine Learning Research*, *26*(204), 1–7. http://jmlr.org/papers/v26/24-1540.html

Egorov, M., Sunberg, Z. N., Balaban, E., Wheeler, T. A., Gupta, J. K., & Kochenderfer, M. J. (2017). POMDPs.jl: A framework for sequential decision making under uncertainty. *Journal of Machine Learning Research*, *18*, 1–5.

Foster, A., Ivanova, D. R., Malik, I., & Rainforth, T. (2021). Deep adaptive design: Amortizing sequential Bayesian experimental design. *Proceedings of the 38th International Conference on Machine Learning*, 3384–3395.

Ghavamzadeh, M., Mannor, S., Pineau, J., & Tamar, A. (2015). Bayesian reinforcement

DRAFT

learning: A survey. *Foundations and Trends in Machine Learning*, *8*(5–6), 359–483. https://doi.org/10.1561/2200000049

Huan, X., Jagalur, J., & Marzouk, Y. (2024). Optimal experimental design: Formulations and computations. *Acta Numerica*, *33*, 715–840. https://doi.org/10.1017/S0962492924000023

Ivanova, D. R., Foster, A., Kleinegesse, S., Gutmann, M. U., & Rainforth, T. (2021). Implicit deep adaptive design: Policy-based experimental design without likelihoods. *Advances in Neural Information Processing Systems*, *34*.

Kleinegesse, S., & Gutmann, M. U. (2020). Bayesian experimental design for implicit models by mutual information neural estimation. *Proceedings of the 37th International Conference on Machine Learning*, 5316–5326.

Rainforth, T., Foster, A., Ivanova, D. R., & Bickford Smith, F. (2024). Modern Bayesian experimental design. *Statistical Science*, *39*(1), 100–114. https://doi.org/10.1214/23-STS915

Salvatier, J., Wiecki, T. V., & Fonnesbeck, C. (2016). Probabilistic programming in Python using PyMC3. *PeerJ Computer Science*, *2*, e55. https://doi.org/10.7717/peerj-cs.55

Segall, M. D. (2012). Multi-parameter optimization: Identifying high quality compounds with a balance of properties. *Current Pharmaceutical Design*, *18*(9), 1292–1310. https://doi.org/10.2174/138161212799436430

Shahriari, B., Swersky, K., Wang, Z., Adams, R. P., & Freitas, N. de. (2016). Taking the human out of the loop: A review of Bayesian optimization. *Proceedings of the IEEE*, *104*(1), 148–175. https://doi.org/10.1109/JPROC.2015.2494218

Sunberg, Z. N., & others. (2024). *MCTS.jl: Monte Carlo tree search in Julia*. https://github.com/JuliaPOMDP/MCTS.jl.

Yang, B., Chung, Y., Yang, A. Y., Yuan, B., Chen, T., & Yu, X. (2025). QComp: A QSAR-based imputation framework for drug discovery. *Journal of Chemical Information and Modeling*, *65*(15), 7862–7873. https://doi.org/10.1021/acs.jcim.5c00059

DRAFT